

C++ Template 延伸自主練習作業

專題名稱：學生成績管理系統



學校：國立台南大學

系級：資工一

學號：S11459004

姓名：翁承佑

GitHub Repository 網址：

<https://github.com/danny540314/cpp-student-management-system>

1. 自主學習主題與目標

本次自主學習的重點在於接續課程 Chapter 16 Templates 的觀念，將其延伸應用至 C++ 標準模板函式庫(STL)。透過兩週的實作，目標是從理論層面理解「template 可以讓函式與類別套用於不同型別」的特性，並在實務上熟悉常見的 STL 容器。此外，透過完成整合應用小專題，不僅能培養依問題需求選擇合適資料結構的能力，也為大二的資料結構課程建立扎實的先備基礎。

2. Template 與 STL 的關係

在 C++ 中，Template 是一種強大的泛型編程工具，允許開發者編寫獨立於特定資料型別的程式碼。而 STL(Standard Template Library)正是 Template 技術的最具體實踐。STL 中常見的容器（如 vector、stack、queue、map 等）與演算法，本質上都是 class template 與 function template 的實際應用。藉由 Template，STL 提供了一套高度可重用、型別安全且效能優異的標準化元件，讓開發者無須從頭打造基礎資料結構，便能專注於高階邏輯的開發。

3. 常見 STL 容器用途整理

透過第 1 週的基礎練習，我總結了幾種常見 STL 容器的底層特性與適用情境：

- **vector(動態陣列)**：支援隨機存取，記憶體空間連續，適合需要頻繁走訪、排序，且不需要在中間頻繁插入或刪除元素的場景。
 - ✓ **底層機制**：連續記憶體配置。當空間不足時，會自動配置一塊更大的記憶體並將舊資料搬移過去。
 - ✓ **時間複雜度**：尾端插入 `push_back()` 通常為 $O(1)$ ，但若觸發重新配置則為 $O(N)$ 。隨機存取 `operator[]` 為 $O(1)$ 。
- **stack(堆疊)**：具有後進先出(LIFO)的特性，非常適合用來處理具有配對關係的問題，例如括號配對檢查或函數呼叫追蹤。
 - ✓ **時間複雜度**：推入 `push()` 與彈出 `pop()` 皆為 $O(1)$ 。
- **queue(佇列)**：具有先進先出(FIFO)的特性，適合用於模擬服務櫃台排隊、廣度優先搜尋(BFS)等需要依序處理任務的情境。
 - ✓ **時間複雜度**：進入隊伍 `push()` 與離開隊伍 `pop()` 皆為 $O(1)$ 。
- **map(關聯式容器)**：以鍵值對(Key-Value)的方式儲存資料，底層通常實作為紅黑樹。其最大的優勢在於能透過唯一的 Key 快速尋找、新增或修改對應的 Value，例如統計單字出現次數或依學號查詢學生資料。
 - ✓ **底層機制**：通常以紅黑樹(Red-Black Tree)這種平衡二元搜尋樹來實作，這保證了容器內的元素會隨時依照 Key 保持排序狀態。
 - ✓ **時間複雜度**：搜尋 `find()`、插入 `insert()` 與刪除的平均時間複雜度皆為 $O(\log N)$ 。

4. 第 2 週小專題設計說明

本週專題為設計一個「學生成績管理系統」，系統的設計需求如下：

- **資料表示**：設計 Student 結構，包含學號(id)、姓名(name) 與成績(score)。
- **容器選擇**：為求查詢效率與防呆機制，本專案選擇 `map<string, Student>` 作為主要儲存容器，並以學號作為 Key。
- **核心功能**：包含新增學生（處理學號之重複）、列出所有學生、依成績由高到低排序、依學號查詢，以及計算平均、最高/低分、及格與不及格人數等統計功能。

5. 主要程式架構與重要程式片段

在本系統中，有幾個關鍵的程式碼實作片段：

a. 自定義 Template Function(固定挑戰)

為了統計最高與最低分，設計了具備泛型特性的比較函式：

```
template <class T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}

template <class T>
T getMin(T a, T b) {
    return (a < b) ? a : b;
}
```

b. 依成績降冪排序

由於 `map` 預設依 Key 排序，我在實作排序功能時，先將資料轉移至 `vector`，並利用 `<algorithm>` 中的 `sort()` 搭配 Lambda 函式進行客製化排序：

```
vector<Student> sortedStudents;
for (const auto& pair : students) {
    sortedStudents.push_back(pair.second);
}

sort(sortedStudents.begin(), sortedStudents.end(), [](const Student& a, const Student& b) {
    return a.score > b.score;
});
```

c. 系統主迴圈與防呆機制架構

為確保管理系統在使用者輸入錯誤（例如預期輸入數字卻輸入英文字母）時不會崩潰或陷入無窮迴圈，我在 switch-case 選單的外層加入了輸入流(input stream)的錯誤偵測與清除機制，這是在開發終端機互動程式時非常重要的穩健性設計：

```
while (true) {
    displayMenu();

    if (!(cin >> choice)) {
        cin.clear();
        cin.ignore(10000, '\n');
        cout << "無效輸入，請重新輸入數字！\n";
        continue;
    }

    if (choice == 0) {
        cout << "系統結束，掰掰！\n";
        break;
    }
}
```

d. 記憶體管理與效能最佳化(Pass by Reference)

在處理 STL 容器與自定義 struct 時，若不注意參數傳遞的方式，很容易造成不必要的記憶體複製，進而拖垮系統效能。在本專題中，我特別在多處採用了「常數參照(Constant Reference)」來進行最佳化：

```
for (const auto& pair : students) {...}
```

```
sort(..., [](const Student& a, const Student& b) {...});
```

6. 執行畫面或輸出結果

- 系統啟動完成畫面(含選擇功能防呆)

命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5):
```

圖 1-1 系統啟動完成

命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): A
無效輸入，請重新輸入數字！
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5):
```

圖 1-2 選擇功能防呆

說明：系統成功擋下錯誤的功能選擇，並重新要求選擇功能。

- 功能 1：新增學生資料(含重複學號防呆)

C:\> 命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 1

[執行] 新增學生資料...
請輸入學號: M001
請輸入姓名: Tomori
請輸入成績: 85
成功新增學生資料!
```

圖 2-1 新增學生成績資料

C:\> 命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 1

[執行] 新增學生資料...
請輸入學號: M002
請輸入姓名: Anon
請輸入成績: 60
成功新增學生資料!
```

圖 2-2 新增學生成績資料

C:\> 命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 1

[執行] 新增學生資料...
請輸入學號: M003
請輸入姓名: Raana
請輸入成績: 98
成功新增學生資料!
```

圖 2-3 新增學生成績資料

C:\> 命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 1

[執行] 新增學生資料...
請輸入學號: M004
請輸入姓名: Soyo
請輸入成績: 75
成功新增學生資料!
```

圖 2-4 新增學生成績資料

C:\> 命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 1

[執行] 新增學生資料...
請輸入學號: M005
請輸入姓名: Taki
請輸入成績: 45
成功新增學生資料!
```

圖 2-5 新增學生成績資料

C:\> 命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 1

[執行] 新增學生資料...
請輸入學號: M003
錯誤: 學號 M003 輸入之學號已存在，請重新確認!
```

圖 2-6 重複輸入學生學號

說明：系統成功擋下重複輸入的學號，並正確寫入新資料。

- 功能 2：列出所有學生

```
Ctrl 命令提示字元 - .\main.exe
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 2

[執行] 列出所有學生...

-----
學號          姓名          成績
-----
M001          Tomori         85
M002          Anon           60
M003          Raana          98
M004          Soyo           75
M005          Taki           45
-----
```

圖 3 列出所有學生資料

說明：透過 map 走訪，資料會自動依照學號(Key)的字典序進行排列。

- 功能 3：依成績降冪排序

```
Ctrl 命令提示字元 - .\main.exe
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 3

[執行] 依成績排序...

-----
學號          姓名          成績
-----
M003          Raana          98
M001          Tomori         85
M004          Soyo           75
M002          Anon           60
M005          Taki           45
-----
```

圖 4 依成績降冪排序學生資料

說明：轉存入 vector 後利用 Lambda 函式成功將成績由高至低印出。

- 功能 4：依學號查詢學生(含不存在學號防呆)

命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 4
[執行] 依學號查詢...
請輸入要查詢的學號: M003
找到學生資料:
學號: M003
姓名: Raana
成績: 98
```

圖 5-1 依學號查詢學生

命令提示字元 - .\main.exe

```
成績: 98
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 4
[執行] 依學號查詢...
請輸入要查詢的學號 M099
找不到學號為 M099 的學生!
```

圖 5-1 依學號查詢學生防呆

說明：運用 `map::find()` 實現 $O(\log n)$ 時間複雜度的快速查找，若查無此人亦有適當錯誤提示。

- 功能 5：全班成績統計

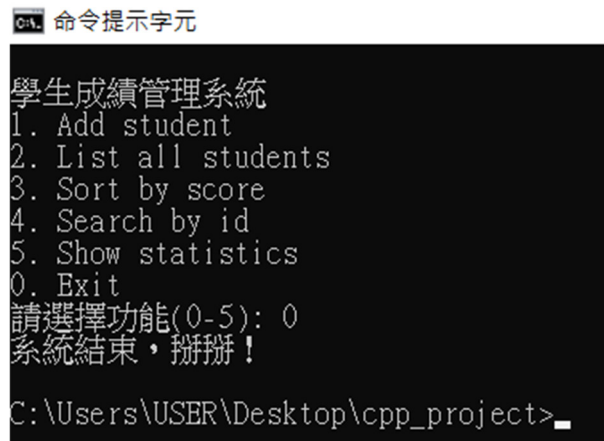
命令提示字元 - .\main.exe

```
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 5
[執行] 顯示成績統計...
總人數: 5 人
全班平均: 72.6 分
最高分: 98 分
最低分: 45 分
及格人數: 4 人
不及格人數: 1 人
```

圖 6 統計成績

說明：精確計算出平均與及格人數，並成功呼叫自定義的 `getMax` 與 `getMin` 樣板函式找出最高與最低分。

- 結束系統



```
命令提示字元
學生成績管理系統
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
請選擇功能(0-5): 0
系統結束，掰掰!
C:\Users\USER\Desktop\cpp_project>
```

圖 7 結束系統

說明：透過選擇功能(0)結束系統。

7. GitHub repo link 與檔案結構說明

- 專案網址：<https://github.com/danny540314/cpp-student-management-system>
- 編譯指令：在 `g++ -std=c++17` 環境下可順利編譯。
- 檔案結構：專案採 Public 權限公開，根目錄包含以下檔案：
 1. main.cpp：主程式源碼。
 2. README.md：包含專案說明與編譯執行步驟。
 3. report.pdf：本份自主學習報告。

8. 遇到的問題與解決方式

在開發與測試過程中，我遇到最深刻的問題是 **Windows 終端機的中文亂碼問題**。

- **問題描述**：原始碼在編輯器中是以 UTF-8 編碼儲存，但 Windows 命令提示字元(cmd) 和 PowerShell 預設使用 Big5 編碼，導致編譯後執行的選單畫面出現「擊貉??」等無法辨識的亂碼。
- **解決方式**：經過查閱資料，我得知這屬於編碼不對稱的問題，我透過在終端機輸入 `chcp 65001` 指令，將執行環境強制切換為支援 UTF-8 編碼的模式。為了讓程式更加完善，後續也可考慮在程式碼中加入 `<cstdlib>` 並呼叫 `system("chcp 65001 > nul")`；讓程式在啟動時自動統一編碼環境，提升跨平台的使用者體驗。

9. 學習心得與未來可延伸方向

在本次的整合專題中，我選擇使用 `map<string, Student>` 作為核心的資料容器。這個決策是基於「學號的唯一性」以及「系統經常需要進行特定人員搜尋」的需求。雖然這使得在實作「依成績排序」時，必須額外花費資源將資料複製到 `vector` 中處理，但綜合考量下，`map` 帶來的 $O(\log n)$ 查詢優勢以及天然的「防範重複 Key 值」特性，大幅簡化了防呆機制與搜尋邏輯的程式碼複雜度，這讓我深刻體會到「根據應用場景選擇合適資料結構」的重要性。

此外，透過實作 `getMax` 與 `getMin`，將原本抽象的 `Template` 語法與具體的成績統計功能結合，真正理解了泛型編程的威力。未來若要延伸此專案，我希望能加入檔案 I/O 功能，讓學生資料能被持久化儲存於 `.csv` 檔中，並嘗試將各項選單功能拆分成不同的函式或標頭檔 (`.h` / `.cpp`)，進一步提升程式的維護性與封裝性。

跨語言的資料結構反思

在學習 C++ STL 的過程中，很容易將 `map` 的使用經驗與開發網頁或應用程式時常用的高階語言特性進行對比。例如在使用 Python 撰寫應用程式時，經常會大量依賴 `Dictionary` 來儲存與傳遞鍵值對資料。

雖然 C++ 的 `map` 與 Python 的 `Dictionary` 在語法操作上感覺非常相似，但透過這次自主學習，我深入了解到它們底層邏輯的巨大差異。C++ 的 `map` 為了維持元素的排序，犧牲了一點點效能，採用紅黑樹實作，查詢時間為 $O(\log N)$ ；而多數高階語言的字典則是採用雜湊表，雖然查詢可達 $O(1)$ ，但內部元素是無序的。若在 C++ 中想要追求純粹的查詢極速且不在乎排序，則應該改用 `<unordered_map>`。這種「知其然，更知其所以然」的底層觀念突破，是這次 C++ 專題帶給我最寶貴的收穫，也讓我未來在面對不同開發環境與框架時，能更精準地評估效能並選擇正確的工具。

10. 邊界條件 (Edge Cases) 測試案例設計

為了確保系統的穩健性 (Robustness)，我在進行功能驗證時，並非隨機輸入資料，而是刻意設計了涵蓋多種「邊界條件」的測試資料集。本系統的測試案例重點包含：

- **極端數值測試**：輸入極高分(如 98 分)與極低分(如 45 分)，用於驗證自訂 `Template Function` (`getMax` 與 `getMin`) 的邊界捕捉能力是否正確。
 - **邏輯邊界測試**：刻意輸入剛好為 60 分的成績。這是及格與不及格的臨界值 (Threshold)，用於驗證系統在 `score >= 60` 條件判斷上是否有遺漏等號的邏輯瑕疵。
 - **例外狀態防禦**：
 1. **重複的唯一鍵值 (Duplicate Key)**：刻意輸入已存在的學號，驗證系統是否能透過 `map::find` 成功攔截並拒絕覆蓋原有資料。
 2. **型別錯誤攻擊**：在提示輸入數字選項時，故意輸入英文字元，驗證 `cin.clear()` 與緩衝區清除機制是否能成功防止程式崩潰。
- ☆ 透過上述測試案例的驗證，證明本系統不僅在正常操作下能順利運行，在面對極端或錯誤輸入時，亦具備良好的自我保護能力。

11. STL 記憶體管理與傳統 C 語言之比較

在過去學習 C 語言時，若要實作動態陣列或鏈結串列 (Linked List)，往往需要手動使用 `malloc` 配置記憶體，並在資料生命週期結束時嚴格呼叫 `free` 來釋放空間，否則極易造成記憶體洩漏 (Memory Leak) 甚至指標錯誤 (Segmentation Fault)。

本次專題全面使用 C++ STL 後，我深刻體會到 RAII (Resource Acquisition Is Initialization) 機制的優勢。無論是 `vector` 還是 `map`，STL 容器皆會在其建構子中自動配置所需的記憶體，並在離開作用域 (Scope) 時觸發解構子自動釋放資源。這意味著我們在撰寫「學生成績管理系統」的各項功能時，完全不需操心底層指標的管理，不僅程式碼變得更加簡潔安全，也大幅降低了除錯的成本，這正是 C++ 演進出 STL 的核心價值之一。